

python code :-

```
import heapq
```

```
graph = {}
```

```
's': [('A', 2), ('B', 1)],
```

```
'A': [('a', 10)],
```

```
'B': [('a', 4)],
```

```
'a': []
```

```
}
```

```
h = {}
```

```
'A': 2
```

```
'B': 3
```

```
'a': 0
```

```
}
```

```
pq = [(h['s'], 0, 's', [])]
```

```
while pq:
```

```
f, g, node, path = heapq.heappop(pq)
```

```
if node == 'a':
```

```
print("Path:", path)
```

```
print("Cost", g)
```

```
break
```



for neighbour cost in graph[node]:

$$\text{new_g} = g + \text{cost}$$

$$\text{new_f} = \text{new_g} + h[\text{neighbour}]$$

heapq.heappush(

A,

(new_f, new_g, neighbour, path + [neighbour])

)

Output

Path: ['S', 'B', 'C']

Cost: 5

Answer 1

How many lines differ from Lab 4

Usually only 2-3 lines change:

Greedy $\Rightarrow$ Priority = $h[\text{neighbour}]$

A* $\Rightarrow$ Priority = $g + \text{cost} + h[\text{neighbour}]$

Q/2) not a bug. A* expands A because its f value looks good at that line.

Q/3) setting $h[B] = 9$ breaks the optimality guarantee because the heuristic overestimates the remaining cost.